KasSigner

Complete User Guide

Air-gapped Offline Signing Device for Kaspa

WARNING: KasSigner is an open-source offline signing device. You are the sole guardian of your keys. No company, no server, no device stores them for you. Manage your funds with care, verify every transaction on screen, and protect your seed phrase as you would protect your sovereignty.

Version 1.0 — March 2026

github.com/InKasWeRust/KasSigner
kassigner@proton.me

Table of Contents

1. What KasSigner Is (and Is Not)
2. Supported Hardware
3. Installing the Build Toolchain
4. Building and Flashing
5. The Home Screen
6. Creating a New Wallet
7. Seed Management (Multi-Slot)
8. Addresses and Keys
9. Signing a Transaction (KSPT)
10. Signing Messages
11. Multisig (M-of-N)
12. BIP85 Child Mnemonics
13. Backup: Encrypted SD Card
14. Backup: Steganographic JPEG
15. Backup: SeedQR (Paper)
16. Importing Seeds and Keys
17. SD Card Management
18. Settings
19. Camera and QR Tips
20. Key Derivation Architecture
21. Security Model
22. Reproducible Builds (Docker)
23. Secure Boot (eFuse)
24. Live Display Mirror
25. KasSee — Watch-Only Companion
26. Troubleshooting
27. Command Reference

1. What KasSigner Is (and Is Not)

KasSigner is an open-source offline signing device built on the ESP32-S3. It generates private keys, signs Kaspa transactions via QR code exchange, and never connects to any network. All key material lives in RAM only and is destroyed on power-off.

What KasSigner IS:

- An **offline signing device** — generates keys, signs transactions, exports via QR
- A **seed generator** — BIP39 mnemonics from hardware entropy or dice rolls
- A **steganographic backup tool** — hides encrypted seeds inside JPEG photos
- **Stateless** — all key material in RAM, destroyed on power-off
- **Open source** — 100% Rust, reproducible Docker builds

What KasSigner is NOT:

- **NOT a hardware wallet** — no secure element, no tamper detection, no persistent storage
- **NOT audited** — no professional security review yet
- **NOT resistant to physical attacks** — JTAG, voltage glitching, flash readers
- **NOT long-term key storage** — your backup IS your storage, not the device

2. Supported Hardware

Why the ESP32-S3?

The ESP32-S3 is a dual-core 240MHz microcontroller with 512KB SRAM, 8MB PSRAM, and built-in USB. It runs bare-metal Rust with no operating system, no network stack, and no background processes. There is nothing between your keys and the silicon. The chip is inexpensive, widely available, and supported by a mature Rust toolchain. Its LCD_CAM peripheral provides native DVP camera input for QR scanning without external ICs. Two I2C buses, SPI, and SDHOST give direct access to display, touch, camera, and SD card — all controlled register-by-register in our code, with no vendor libraries or binary blobs in the signing path.

	Waveshare ESP32-S3-Touch-LCD-2	M5Stack CoreS3 / CoreS3 Lite
Price	~$25	~$45
Display	ST7789T3 320x240	ILI9342C 320x240
Camera	OV5640 5MP autofocus	GC0308 QVGA
Touch	CST816D capacitive	FT6336U capacitive
SD Card	SDHOST native (fast)	Bitbang SPI
Audio	—	AW88298 speaker
PSRAM	8MB octal	8MB octal

Waveshare pin map:

```
Camera: XCLK=8 PCLK=9 VSYNC=6 HREF=4 D0-D7=12,13,15,11,14,10,7,2
Camera I2C1: SDA=21 SCL=16 PWDN=17
Display SPI: MOSI=38 SCLK=39 CS=45 DC=42 RST=0 BL=1
Touch I2C0: SDA=48 SCL=47 INT=46
SD SDHOST: CLK=39 CMD=38 D0=40 | Battery: ADC=GPIO5
```

M5Stack CoreS3 pin map:

```
Camera: XCLK=2 PCLK=13 VSYNC=42 HREF=41
D0-D7=15,16,48,12,10,40,39,11
Display SPI2: MOSI=37 SCLK=36 CS=3 DC=35
Touch I2C0: SDA=12 SCL=11 INT=21
SD SPI: MOSI=37 SCLK=36 CS=4 (bitbang, shared bus)
PMU: AXP2101+AW9523B I2C0 | Speaker: AW88298 I2S
```

3. Installing the Build Toolchain

Step 1: Install Rust

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env
```

Step 2: Install espup (Xtensa toolchain)

```
cargo install espup
espup install
```

Step 3: Install espflash

```
cargo install espflash
```

Step 4: Clone and verify

```
git clone https://github.com/InKasWeRust/KasSigner.git
cd KasSigner/tools
cargo run --bin kassigner-setup
```

> The setup checker verifies Rust, espup, Xtensa toolchain, espflash, Docker, and serial ports. It shows exact install commands for anything missing.

4. Building and Flashing

Step 1: Connect board via USB-C and verify serial port

```
# macOS:
ls /dev/cu.usbmodem*

# Linux:
ls /dev/ttyUSB* /dev/ttyACM*
```

Step 2: Build and flash

Waveshare:

```
cd bootloader
ESP_HAL_CONFIG_PSRAM_MODE=octal cargo run --release
```

M5Stack:

```
cd bootloader
cargo run --release --no-default-features --features m5stack
```

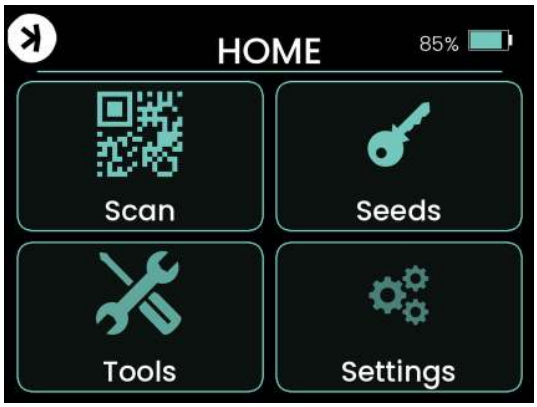
First build takes ~2-3 minutes (compiles all dependencies). Subsequent builds take ~5-10 seconds. The device reboots automatically after flashing and shows the home screen.

> To skip hardware self-tests for faster iteration, add **--features skip-tests** to your build command.

5. The Home Screen

After boot, KasSigner shows a 2x2 grid with four main areas:

- **Scan** — scan QR codes (KSPTs, SeedQR, pubkeys for multisig)
- **Seeds** — manage seed slots, view addresses, export keys
- **Tools** — create seeds, import keys, BIP85, multisig, stego, message signing
- **Settings** — display brightness, SD card, about screen



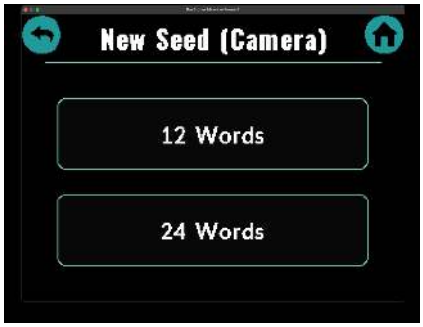
Tap any of the four cards to enter that section. The Kaspa logo and battery indicator are shown on the home screen only. Inside any menu, tap the top-left corner to go back.

6. Creating a New Wallet

From the home screen, tap **Tools** to access seed creation methods.

Method A: Hardware Random (TRNG)

- Tap **New Seed** → choose 12 or 24 words
- Words generated from ESP32-S3 hardware TRNG + camera noise + ADC entropy
- Words shown one at a time — swipe right to advance



WARNING: Write down every word on paper. These words ARE your wallet. Do not photograph. Do not type into any internet-connected device.

Method B: Dice Rolls (verifiable entropy)

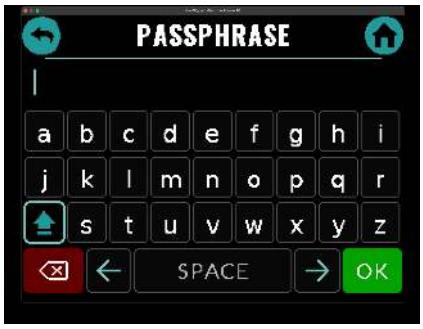
- Tap **Dice Seed** → choose 12 or 24 → roll a physical die 99 or 198 times
- Tap the die face (1-6) after each roll. Fully verifiable — same rolls produce the same mnemonic in any BIP39 tool

Method C: Calculate Last Word

• Tap **Calc Last Word** → enter the first 11 or 23 words → device computes the valid checksum word

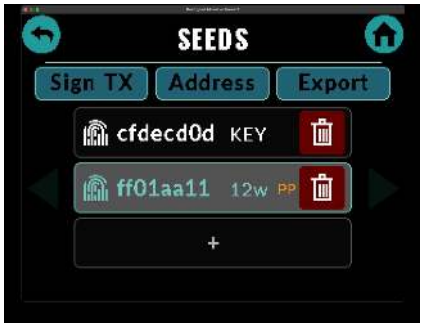
BIP39 Passphrase (25th word)

After seed creation, you can add an optional passphrase. This creates a hidden wallet: the 12/24 words alone lead to a decoy wallet, while words + passphrase lead to the real wallet. Keep your passphrase safe — it is never stored on the device.



7. Seed Management (Multi-Slot)

KasSigner holds up to 16 seed slots in RAM. Nothing is persisted — all slots are lost on power-off.



- Each slot shows a **fingerprint** hex code, the slot type (**KEY**, **12w**, **24w**), and **PP** if a passphrase is active
- The **active slot** is highlighted in teal — this is the one used for signing and address display
- Tap any slot to activate it
- Red **trash** icon securely zeroizes and removes a slot from RAM
- Empty **+** slot: tap to go to Tools for seed creation or import
- Three action buttons at the top: **Sign TX**, **Address**, **Export** — these operate on the active slot

Slot types:

- **Mnemonic** (12w / 24w) — full capabilities: derive addresses, sign, BIP85, SeedQR, kpub, xprv
- **XPrv** — account-level key: derive addresses, sign, kpub. No BIP85 or SeedQR
- **Raw Key** (KEY) — single address only: sign with one key, no derivation tree

8. Addresses and Keys

Viewing addresses:

From the Seeds screen, tap **Address** to see the receive address for the active seed at the current derivation index.

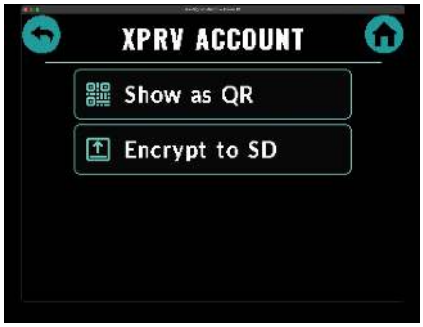
- Tap the address text to display it as a full-screen QR code
- Use the **<** and **>** buttons to navigate between address indexes
- Tap the **#N** button to jump to a specific index using the keypad

Exporting keys:

From the Seeds screen, tap **Export** to open the Export Wallet menu.

- **Show Seed Words** — display your BIP39 mnemonic words on screen
- **QR Export** — submenu: CompactSeedQR, Standard SeedQR, Plain Words QR
- **JPEG Stego Export** — hide your seed inside a JPEG photo on SD card
- **kpub Watch-Only** — display the extended public key as QR (safe to share)

- **xprv Account** — submenu: show the account key as QR or encrypt it to SD card



- **Seed Backup to SD** — encrypt seed words with a password and save to SD card

> The kpub allows viewing all addresses and balances but CANNOT sign transactions. Safe to import into an internet-connected companion wallet for building transactions.

9. Signing a Transaction (KSPT)

Why KSPT? Our custom binary format

Kaspa does not have a standard for transmitting unsigned transactions via QR code. The ecosystem uses PSKB (Partially Signed Kaspa Binary), a JSON-based format that encodes everything as hex strings. While universal, PSKB is extremely verbose — a simple 1-input, 2-output transaction becomes over 2,000 bytes in JSON, requiring 9 QR frames to scan.

KasSigner uses **KSPT** (KasSigner Packed Transaction), a custom compact binary format designed specifically for air-gapped QR transport. KSPT strips all JSON overhead, encodes values as raw bytes, and produces payloads 10-15x smaller than PSKB. A transaction that needs 9 QR frames in PSKB fits in a single QR with KSPT.

KSPT is not a Kaspa standard — it is a KasSigner-specific format understood by the device and its companion app KasSee. The companion translates between the Kaspa network's native format and KSPT for QR transport.

Format	1in/2out	Frames	3in/4out	Frames	QR prefix
Standard PSKB hex(JSON), all fields	2,016 B	9	4,512 B	19	PSKB
Opt 2: raw JSON strip nulls, no hex	675 B	3	1,503 B	7	PSKB
Opt 3: compressed strip + zlib + binary QR	330 B	2	338 B	2	PSKZ
Opt 1: KSPT binary custom compact format	161 B	1	361 B	2	KSPT

Opt 1: KSPT binary

+ Smallest payload (1 QR simple)
+ Device already parses it
+ Zero firmware changes
+ Instant scan, best UX

– Custom format, only KasSigner
– Two formats to maintain
– No covenants/DarkNight support
– Must sync if PSKB evolves

Opt 2: raw JSON

+ Standard PSKB JSON inside
+ No extra dependencies
+ Any PSKB parser reads it
+ Simple implementation

– 3 frames simple, 7 complex
– JSON parser needed on ESP32
– Scales poorly with inputs
– Verbose field names waste bytes

Opt 3: compressed

+ 2 frames even for complex TX
+ Standard PSKB after decompress
+ Future-proof (covenants++)
+ Scales perfectly (repetitive JSON)

– Adds miniz_oxide (~15KB flash)
– Custom PSKZ prefix
– More processing on device
– Can't eyeball compressed data

Air-gapped signing workflow

The device never connects to any network — data exchange happens via QR codes or hex text over serial. You can also import a seed from SD card or type an address manually.

> **Before signing, make sure you have loaded a seed or imported a key into a slot. Without an active seed, the signing process will fail.**

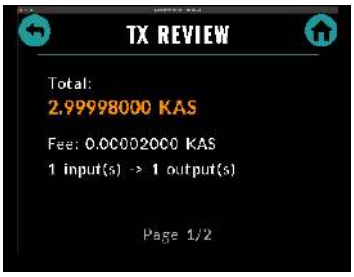
1. KasSee builds an unsigned KSPT and displays it as an animated QR.
2. KasSigner scans the QR, parses the transaction, and shows a review screen.
3. You verify the destination, amount, and fee on the device screen.
4. The device signs each input and displays the signed KSPT as a QR. The signed hex is also output over serial.
5. KasSee scans the signed QR or receives the hex text, and broadcasts to the Kaspa network.

Scan the unsigned KSPT

From the home screen, tap **Scan**. Point the camera at the QR code displayed by KasSee. For multi-frame QR codes, hold steady — the device detects and decodes each frame automatically.

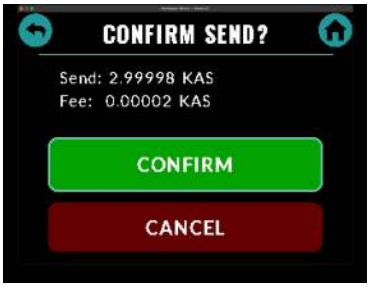
Review the transaction

The device shows total amount, fee, number of inputs/outputs, and the destination address for each output:



Confirm or cancel

Tap **CONFIRM** to sign or **CANCEL** to abort.



Signing

The device derives the correct private key for each input (searching both receive and change chains), computes the sighash using keyed Blake2b-256, and signs with Schnorr. For multi-input transactions, ~3 seconds per input.

Export signed transaction

The device displays the signed KSPT as a QR code. The signed hex is also output as a single line over serial for copy-paste.

Broadcast

In KasSee, scan the signed QR or paste the hex text:

```
kassee broadcast <signed_hex>
```

```
0000000000000000000000000000000000000000000000000000000000000000f20cbd7899a567079649654c07235
6618bead9769708bf28c0a7f29765c936d20000000000a3e111000000000000000000000000000100002220
8ce4daaa503c0e74f6161179aac099ca3c2674f91e3f401d468e8aac48917ef1ac40dfeed335c793eb786
b4a593355a11ae91bc999facde10e478f84f400928bf142a363b2bd4b7313c02823e726926017655267bb
708de8f627e9f37bc1b3443fc901309be111000000000000002220435dcc6fd21e1e59d78677fd1cf56aff0
b14e60a2ec2c038f1e32cfdde1aa002ac
  Parsing signed KSPT v1, 249 bytes
  TX: v0, 1 inputs, 1 outputs
  Input 0: signed (64 byte sig, sighash=0x1)
  TX ID: 2faa58b247d8bf5b8f8d3d2a467b6821738e083ff7fcad06dfeffb09c520046d
  TX version: 0
  Inputs: 1
    Input 0: sig_script 66 bytes = 41dfeed335c793eb786b4a593355a11ae91bc999facde10e47
8f84f400928bf142a363b2bd4b7313c02823e726926017655267bb708de8f627e9f37bc1b3443fc901
  Outputs: 1
  Connecting to Kaspa network...
  Connected!
Transaction broadcast!
  TX ID: 2faa58b247d8bf5b8f8d3d2a467b6821738e083ff7fcad06dfeffb09c520046d
```

The first air-gapped hardware-signed Kaspa transaction was broadcast on mainnet on March 30, 2026 using KasSigner. TX ID: 2faa58b247d8bf5b8f8d3d2a467b6821738e083ff7fcad06dfeffb09c520046d

Transaction size limits

The current signed transaction buffer is 1024 bytes. Maximum capacity:

- 2 outputs: max 5 inputs
- 1 output (no change): max 6 inputs

If your wallet has many small UTXOs, compound them in batches of 3-5.

Transaction data capacity will be increased in future firmware versions with the PSKZ compressed transport format, enabling 8+ inputs per transaction.

10. Signing Messages

KasSigner can sign arbitrary text messages with any address key. This proves you control an address without making a transaction.

How it works

From the Tools menu, tap **Sign Message**.

- Choose **Type manually** to enter text using the on-screen keyboard
- Or choose **Load .TXT from SD** to sign the contents of a text file on SD card

The device hashes the message with SHA-256 and signs the hash with Schnorr using the active address key. The result is displayed as a QR code containing:

- The original message text
- The Kaspa address that signed it
- The 64-byte Schnorr signature

Anyone can verify the signature against the address to confirm the signer controls the private key behind that address.

11. Multisig (M-of-N)

KasSigner supports M-of-N multisig addresses. Multiple co-signers each contribute a public key, and M of the N keys must sign to spend funds.

Creating a multisig address

From the Tools menu, tap **Multisig**. Set M (required signatures) and N (total keys) using the +/– buttons, then tap **NEXT**.

For each key position, choose **Scan kpub QR** (scan another signer's public key) or **Use Loaded Seed** (use a seed already in a slot).

When using a loaded seed, tap the slot you want to contribute as a key:

Multisig wallet address

After all N keys are added, the device derives and displays the multisig address. Tap the address for a full-screen QR.

The descriptor screen shows all participating public keys and can be exported to SD card for backup:

Multisig co-signing and multisig transaction support in KasSee are planned for a future version. Currently, multisig address creation and descriptor export work on the device.

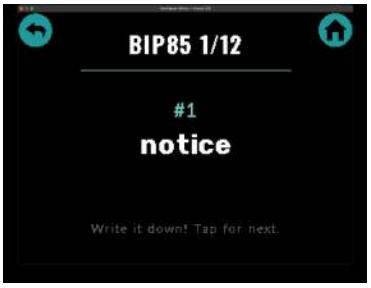
12. BIP85 Child Mnemonics

BIP85 derives independent child mnemonics from a master seed. Each child is a complete wallet that cannot be linked back to the parent. Use this to create separate wallets for different purposes from one master backup.

From the Tools menu, tap **BIP85 Child**. Choose 12 or 24 words:

Set the child index (0, 1, 2, ...) using +/– buttons, then tap **DERIVE**:

The child mnemonic is displayed word by word. Write each word down — tap for next:



Each index produces a different child. Index 0 always produces the same child from the same master. Write down the child words separately — they are a complete, independent wallet.

13. Backup: Encrypted SD Card

KasSigner can encrypt your seed words with AES-256-GCM and save them to a MicroSD card. The encryption key is derived from a password you choose via PBKDF2-HMAC-SHA512.

Creating a backup

- Insert a FAT32-formatted MicroSD card
- From the Seeds screen, tap **Export** → **Seed Backup to SD**
- Enter a password on the keyboard and tap **OK**
- The device encrypts and writes the backup file (progress bar shown)
- File is named with the seed fingerprint (e.g. **SDC0B1**)

Restoring from backup

- From the Tools menu (page 2), tap **Import from SD**
- The device detects backup files and shows a file list
- Files matching a loaded seed show a label (e.g. "Seed #2")
- Tap a file, enter the password, and the seed is decrypted and loaded
- Progress bar: 0–80% PBKDF2 derivation, 90% decrypt, 100% loaded

Deleting a backup file

- Tap the red trash icon on a file card → confirmation screen
- **CANCEL** (teal outline, left) or **DELETE** (red, right)
- Tap DELETE → button changes to "HOLD 4s" → press and hold 4 seconds to confirm
- Release early or move finger off the button to cancel

If you forget the password, the backup cannot be recovered. There is no reset mechanism. Keep your password safe.

14. Backup: Steganographic JPEG

KasSigner's steganographic backup hides your seed inside an ordinary JPEG photograph. Three layers of protection:

- **Layer 1 — Steganography:** The encrypted seed is stored in JPEG EXIF metadata. The image looks ordinary. Nobody knows which file matters.
- **Layer 2 — Encryption:** The seed is encrypted with AES-256-GCM. The key is derived via PBKDF2 from the image descriptor text you enter.
- **Layer 3 — BIP39 passphrase:** Even if decrypted, the 24 words alone lead to a decoy wallet. The real wallet lives behind a passphrase only in your memory.

Creating a stego backup

From Seeds → Export → **JPEG Stego Export**. Select a JPEG from the SD card:

Enter or load the image descriptor — this text becomes the encryption key:

Optionally hide a recovery hint for your BIP39 passphrase inside the EXIF data. The answer to the hint IS your passphrase:

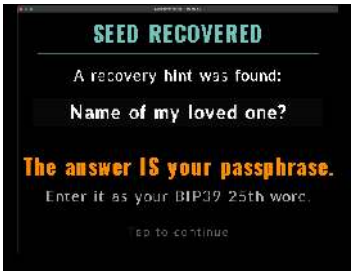
Confirm the overwrite — the original JPEG is modified in place with the encrypted seed embedded:

Restoring from stego backup

From the Tools menu (page 3), tap **Stego Import**:

Enter the exact image descriptor used during backup:

If a recovery hint was stored, the device shows it. The answer IS your BIP39 passphrase — enter it as the 25th word:



> The descriptor must match exactly — including capitalization, spaces, and punctuation. There is no recovery mechanism if forgotten. For a deep dive into the cryptographic design, threat model, and why this approach works, read **docs/STEGANOGRAPHY.md** in the repository.

15. Backup: SeedQR (Paper)

SeedQR encodes your BIP39 mnemonic as a QR code that you print on paper or engrave on metal. It is a community-standard format compatible with SeedSigner, Krux, and other air-gapped devices.

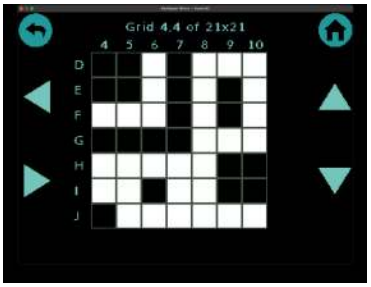
KasSigner supports three SeedQR variants:

- **CompactSeedQR** — binary encoding of BIP39 word indices. Smallest QR, fastest scan. Recommended.
- **Standard SeedQR** — 4-digit zero-padded BIP39 indices. Compatible with SeedSigner.
- **Plain Words QR** — mnemonic words as plain text. Human-readable but largest QR.

Exporting a SeedQR

From Seeds → Export → **QR Export**:

The device displays the QR on screen. Tap for the grid view to hand-copy the pattern onto paper:



Importing a SeedQR

From the home screen, tap **Scan**. Point the camera at any SeedQR (compact, standard, or plain). The device auto-detects the format, decodes the mnemonic, and loads it into a seed slot.

SeedQR is unencrypted — anyone who sees or photographs the QR has your seed. Treat SeedQR backups with the same care as written seed words.

SeedQR only stores the mnemonic words. It does not include your BIP39 passphrase (25th word). **Always use a passphrase for real security** — the passphrase turns your seed into a completely different wallet that cannot be derived from the words alone.

16. Importing Seeds and Keys

KasSigner supports multiple import methods to load seeds and keys from external sources into a slot.

Import via QR scan

From the home screen, tap **Scan**. The device auto-detects the QR type:

- **CompactSeedQR** — loads 12 or 24 word mnemonic
- **Standard SeedQR** — 4-digit BIP39 indices
- **Plain Words QR** — space-separated mnemonic text
- **xprv QR** — account-level extended private key
- **kpub QR** — watch-only extended public key (for multisig)

Import words manually

From Tools → **Import Words**. Enter each BIP39 word using the on-screen keyboard with autocomplete suggestions. After 12 or 24 words, the device validates the checksum and loads the seed.

Import from encrypted SD card

From Tools → **Import from SD**. See chapter 13 for details.

Import from steganographic JPEG

From Tools → **Stego Import**. See chapter 14 for details.

Import raw private key

From Tools → **Import Raw Key**. Enter a 32-byte hex private key via QR scan or the keyboard. This creates a single-address KEY slot — no derivation tree, just one address and one key.

> After any import, you are prompted for an optional BIP39 passphrase (25th word). This applies to mnemonic imports only — raw keys and xprv imports skip this step.

17. SD Card Management

KasSigner uses MicroSD cards for encrypted seed backups, steganographic JPEG backups, multisig descriptors, and text file loading. From Settings → **SD Card** you can check the card status, format it, or test read/write:

Requirements

- **FAT32** format — the device reads/writes FAT32 only. exFAT and NTFS are not supported.
- MicroSD up to 32GB recommended. Larger cards may work if formatted as FAT32.
- Standard speed cards are fine — data volumes are small (backup files are under 1KB).

File types on SD

- **.KSS** — encrypted seed backup (AES-256-GCM). Named by fingerprint (e.g. SDC0B1).
- **.JPG** — JPEG photos (for stego backup). Device reads/writes EXIF metadata.
- **.TXT** — text files (for message signing or descriptor loading).
- **.KSDESC** — multisig descriptor files.

Troubleshooting

- **SD not detected:** Remove and reinsert. Ensure FAT32 format. Try a different card.
- **File not showing:** Filenames must be 8.3 format (uppercase, no long names).
- **Write error:** Check the card isn't write-protected (physical lock tab on adapter).

> The SD card is never required for basic operation. Seeds can be created, imported, and used for signing without an SD card. The card is only needed for encrypted backups, stego backups, and message signing from file.

18. Settings

From the home screen, tap **Settings** to access device configuration.

Display

Adjusts the LCD backlight level. Higher brightness improves QR scanning by the companion camera but draws more power from battery or USB.

Audio

Sound settings (M5Stack only — the Waveshare board has no speaker). Toggle confirmation beeps and adjust volume.

SD Card

Shows card status (detected/not detected, type, capacity). Provides **Format FAT32** to wipe and reformat the card on-device, and **Test R/W** to verify read/write. See chapter 17 for details.

About

Shows firmware version, build hash, hardware target, and project information.

19. Camera and QR Tips

The OV5640 (Waveshare) and GC0308 (M5Stack) cameras decode QR codes in software using a custom decoder written in Rust. No external libraries. Here are tips for reliable scanning.

Lighting

- Bright, even lighting works best. Avoid shadows falling across the QR code.
- The screen displaying the QR should be at maximum brightness.
- Avoid direct sunlight hitting the camera lens — causes washout.

Distance and angle

- Hold the device 10–15 cm from the QR code for best focus.
- Keep the camera parallel to the QR — avoid extreme angles.
- For small QR codes (SeedQR), move closer. For large codes (multi-frame), pull back slightly.

Multi-frame QR codes

- KasSee displays animated QR frames that cycle automatically.
- Hold the camera steady — the device detects 0-based frame numbers and assembles them.
- The red border turns green on the sides as frames are captured. All green = complete.
- If a frame is missed, just keep holding — the animation loops and the device catches up.

OV5640 thermal note (Waveshare)

The OV5640 sensor runs warm during continuous capture. After extended use (5+ minutes of scanning), image quality may degrade slightly. This is normal. The camera recovers after a brief rest. Do not modify OV5640 registers to try to reduce heat — any register write kills VSYNC and requires a full power cycle to recover.

> If the camera seems stuck or shows a frozen image, unplug USB, wait 10 seconds, and reconnect. This performs a full OV5640 power cycle.

20. Key Derivation Architecture

KasSigner follows the BIP32/BIP39/BIP44 key derivation hierarchy used by Kaspa wallets. Understanding this tree is important for verifying that your addresses match between KasSigner and other Kaspa software.

Derivation path

```
m / purpose' / coin_type' / account' / change / index
m / 44' / 111111' / 0' / 0 / 0
```

- **purpose = 44'** — BIP44 standard
- **coin_type = 111111'** — Kaspa's registered coin type
- **account = 0'** — first account (hardened)
- **change = 0** — receive chain (external). Change chain = 1.
- **index = 0, 1, 2...** — sequential address index

Key levels

- **Master key (m)** — derived from BIP39 seed. Only exists with full mnemonic.
- **Account key (m/44'/111111'/0')** — this is the xprv level. Can derive all addresses and sign, but cannot derive BIP85 children.
- **Address key (m/.../0/N)** — individual receive address. Single secp256k1 keypair.
- **kpub** — the extended public key at account level. Derives all public addresses but cannot sign. This is what KasSee imports.

Change addresses

When a transaction doesn't spend the full UTXO, the remainder goes to a change address on the change chain (m/.../1/N). KasSigner searches both receive (chain 0) and change (chain 1) when signing, so it correctly signs inputs from either chain.

21. Security Model

KasSigner's security is based on air-gap isolation and ephemeral key storage. It is not a hardware wallet with a secure element — it is a signing device that makes deliberate tradeoffs.

What KasSigner protects against

- **Network attacks:** No WiFi, no Bluetooth, no radio. The ESP32-S3's radio is disabled in hardware (eFuse) when Secure Boot is enabled. There is no code path that initializes any wireless peripheral.
- **Malware on your computer:** The device never sends keys over USB. Only signed transaction hex flows out over serial. A compromised computer cannot extract keys.
- **Memory persistence:** All key material lives in SRAM only, destroyed on power-off. On panic, the firmware wipes the entire 512KB SRAM (two passes) before halting.
- **Supply chain (with Secure Boot):** Secure Boot V2 ensures only firmware signed with your key can run. See chapter 23.

What KasSigner does NOT protect against

- **Physical access attacks:** JTAG, voltage glitching, flash readers, cold boot attacks on SRAM. The ESP32-S3 has no tamper-detect hardware.
- **Side-channel attacks:** No constant-time guarantees on all cryptographic paths (though hash comparison uses constant-time). Power analysis and EM emanation attacks are possible.
- **Evil maid:** If someone swaps your device for a modified one, you wouldn't know (unless Secure Boot is enabled and you verify the firmware hash).
- **Firmware bugs:** The code has not been professionally audited.

Defense in depth

KasSigner's real security comes from layering multiple independent protections:

- **Air gap:** No network connection → no remote attack surface.
- **Ephemeral RAM:** Keys exist only while powered → no persistent key storage to attack.
- **BIP39 passphrase:** Even with the 24 words, the real wallet is behind a passphrase only in your head.
- **Steganographic backup:** Your seed backup looks like a photo. Nobody knows which file matters.
- **Open source:** Every line of code is auditable. No vendor libraries in the signing path.
- **Reproducible builds:** Docker builds produce bit-identical binaries. You can verify the firmware matches the source.
- **Secure Boot V2:** Only your signed firmware runs on the device.

Panic wipe

If the firmware encounters an unrecoverable error (panic), it immediately wipes the entire 512KB SRAM using two full passes of **write_volatile** before halting. This ensures no key material survives in memory after a crash, even if the device is powered and physically captured.

KasSigner has NOT been professionally audited. Use it with full awareness that bugs may exist. Your ultimate protection is the BIP39 passphrase — a secret that exists only in your memory and never touches any device.

22. Reproducible Builds (Docker)

KasSigner provides a Docker-based reproducible build system. Anyone can build the firmware from source and verify the binary matches the official release — bit for bit. Docker runs on macOS, Linux, and Windows.

Prerequisites: Install Docker

Download and install Docker Desktop from **https://docker.com/get-started**. Available for macOS, Windows, and Linux. Verify it's running:

```
docker --version
```

Step 1: Build the toolchain base image (once)

```
docker build --platform linux/amd64 \
-f Dockerfile.base -t kassigner-toolchain:v1 .
```

Creates a frozen x86_64 image with Rust 1.84.0, espup 0.16.0, and the Xtensa toolchain. Identical on every machine regardless of host OS or architecture.

Step 2: Build the firmware (both platforms)

```
docker build --platform linux/amd64 -t kassigner-build .
```

Compiles both Waveshare and M5Stack binaries inside the container, runs **objcopy** to extract clean .bin files, and prints their SHA-256 hashes.

Step 3: Verify

```
docker run --rm kassigner-build
# === Waveshare ===
# <sha256> /build/kassigner-waveshare.bin
# === M5Stack ===
# <sha256> /build/kassigner-m5stack.bin
```

Compare these hashes with the ones in the GitHub Release. If they match, the binary was built from the exact same source.

Why it works

- **Forced x86_64** — identical hashes on Intel and Apple Silicon
- **Pinned toolchain** — exact Rust and espup versions frozen in base image
- **Cargo.lock** — every dependency pinned to exact version and hash
- **SOURCE_DATE_EPOCH=0** — eliminates timestamps from the build
- **objcopy** — strips build-path metadata from final binary

> Reproducible builds let you verify that the firmware running on your device matches the public source code. Don't trust — verify.

23. Secure Boot (eFuse)

> **ALL eFuse operations are IRREVERSIBLE. A mistake can permanently brick the board. Read the full EFUSE_RUNBOOK.md in the docs/ folder before touching any commands. Test on a sacrificial board first.**

KasSigner has two independent signature verification layers: hardware Secure Boot V2 (RSA-3072, verified by the silicon ROM) and a software Schnorr check (verified by our code). Both must pass for the firmware to run.

Step 0: Verify clean board

```
espefuse.py --port /dev/tty.usbmodem* --chip esp32s3 summary
```

All security eFuses must be at default (False/0). Stop if any are already set.

Step 1: Generate RSA-3072 signing key

```
espsecure.py generate_signing_key --version 2 \
--scheme rsa3072 secure_boot_v2_key.pem
```

Back this up to 3+ offline locations immediately. Losing this key bricks the board forever.

Step 2: Generate and burn key digest

```
espsecure.py digest_sbv2_public_key \
--keyfile secure_boot_v2_key.pem --output digest0.bin

espefuse.py --port /dev/tty.usbmodem* --chip esp32s3 \
burn_key BLOCK_KEY0 digest0.bin SECURE_BOOT_DIGEST0
```

Step 3: Revoke unused key slots

```
espefuse.py --port ... burn_efuse SECURE_BOOT_KEY_REVOKE1
espefuse.py --port ... burn_efuse SECURE_BOOT_KEY_REVOKE2
```

Step 4: Sign and flash BEFORE enabling secure boot

```
espsecure.py sign_data --version 2 \
--keyfile secure_boot_v2_key.pem \
--output kassigner-signed.bin kassigner-bootloader.bin

espflash flash --chip esp32s3 kassigner-signed.bin
```

Verify it boots correctly before proceeding.

Step 5: Enable Secure Boot (point of no return)

```
espefuse.py --port /dev/tty.usbmodem* --chip esp32s3 \
burn_efuse SECURE_BOOT_EN
```

After this, only firmware signed with your key will boot.

Step 6: Production lockdown (optional)

For maximum security, disable debug interfaces:

```
espefuse.py --port ... burn_efuse DIS_PAD_JTAG
espefuse.py --port ... burn_efuse DIS_USB_JTAG
espefuse.py --port ... burn_efuse DIS_USB_SERIAL_JTAG
espefuse.py --port ... burn_efuse DIS_DIRECT_BOOT
espefuse.py --port ... burn_efuse ENABLE_SECURITY_DOWNLOAD
```

Do NOT burn **DIS_DOWNLOAD_MODE** unless absolutely sure — this permanently prevents any firmware update via UART. Since KasSigner has no OTA, this makes the board unupdatable.

What gets disabled

- **JTAG** — hardware debugging port permanently disabled
- **USB Serial/JTAG** — debug serial interface disabled
- **Direct boot** — forces ROM to always verify signatures
- **Unsigned firmware** — only signed binaries boot

Recovery

There is no recovery from a bricked eFuse configuration. If secure boot is enabled and the signing key is lost, the board is permanently bricked. Buy a new board.

> The complete step-by-step process with pre-flight checklist, flash encryption options, eFuse budget table, and decision matrix is in **docs/EFUSE_RUNBOOK.md**.

24. Live Display Mirror

KasSigner can mirror its display to a computer in real-time over the USB-JTAG serial interface. This creates a window on your computer showing exactly what the device screen shows — useful for demos, screenshots, and troubleshooting. You need **two terminal windows**.

Setup: two terminals

Terminal 1 — Flash and run the firmware:

From the **bootloader/** directory, build with the mirror feature and flash:

```
cd bootloader
ESP_HAL_CONFIG_PSRAM_MODE=octal cargo run --release \
--features waveshare,mirror,skip-tests
```

Or flash a pre-built binary:

```
espflash flash \
target/xtensa-esp32s3-none-elf/release/kassigner-bootloader
```

Terminal 2 — Launch the mirror viewer:

From the **tools/** directory, run the mirror tool pointing to the JTAG serial port:

```
cd tools
cargo run --release --bin kassigner-mirror -- /dev/cu.usbmodem21201
```

Replace **/dev/cu.usbmodem21201** with your actual JTAG-serial port. A 960x720 window opens showing the device screen in real-time.

Features

- Real-time framebuffer streaming over JTAG-serial at ~15 fps
- 960x720 window using **minifb** (pure Rust, no SDL)
- Arrow key navigation from the computer keyboard
- PNG export with **--export** flag for automated screenshots

> The mirror is a development/demo tool. It does not transmit any key material — only pixel data flows over serial. The device operates identically with or without the mirror connected.

25. KasSee — Watch-Only Companion

KasSee is a standalone Rust CLI application that runs on your internet-connected computer. It holds only the extended public key (kpub) — it can see your addresses and balances but cannot sign transactions. "See" = watch-only. (Also a Spanish pun: "Casi" = almost a wallet.)

Setup

```
cd kassee
cargo build --release
./target/release/kassee import <kpub_string>
```

The kpub is obtained from KasSigner: Seeds → Export → kpub Watch-Only. Scan the QR or copy the hex string.

Commands

```
kassee balance # show balance
kassee addresses -n 10 # show receive addresses
kassee addresses --change -n 5 # show change addresses
kassee send <addr> <amount> # build unsigned KSPT
kassee send <addr> <amount> --fee 20000
kassee broadcast <signed_hex> # broadcast signed TX
```

Features

- Connects to Kaspa mainnet via wRPC (Rusty Kaspa SDK)
- Fee estimation via **get_fee_estimate()** RPC
- Storage mass awareness (KIP-9): warns below 0.2 KAS, errors below 0.1 KAS
- Address reuse detection with warning
- Change address auto-rotation
- Balanced multi-frame QR output as animated GIF
- Wallet state stored in **kassee.json** (local file)

> KasSee never has access to private keys. It builds unsigned transactions that must be signed by KasSigner. The signed transaction is returned via QR or hex text for broadcast.

26. Troubleshooting

Device won't boot / blank screen

- Unplug USB, wait 10 seconds, reconnect. This forces a full power cycle.
- Try a different USB cable — some cables are charge-only (no data).
- Hold the BOOT button while plugging in to enter ROM download mode, then reflash.

Camera frozen / no image

- The OV5640 requires a hard power cycle if registers were corrupted. Unplug USB, wait 10 seconds, reconnect.
- Check lighting — very dark environments may show a near-black image.
- On M5Stack, the GC0308 is lower resolution. Move the QR closer to the camera.

QR not scanning

- Increase display brightness on the device showing the QR.
- Hold 10–15 cm from the QR, parallel to the screen.
- For multi-frame, hold steady and wait for all frames to cycle.
- Avoid reflections on glass screens — tilt slightly if needed.

SD card not detected

- Must be FAT32 format. Cards over 32GB need manual FAT32 formatting.
- Remove and reinsert firmly.
- Use Settings → SD Card → Format FAT32 to format on-device.
- Try a different MicroSD card.

Signing failed

- Make sure the correct seed is loaded and active (check fingerprint).
- The transaction must be addressed to keys derivable from the active seed.
- Check "Too many inputs" error — maximum is 5 inputs with 2 outputs.

Wrong password on SD import

- The error message says "Wrong password" — re-enter carefully.
- Passwords are case-sensitive. Check exact capitalization.

Touch not responding

- KasSigner uses tap-on-lift (fires on finger release, not press). Tap and release cleanly.
- The CST816D (Waveshare) enters auto-sleep after idle. A first tap wakes it — the second tap registers.
- Avoid tapping with wet fingers or through screen protectors.

Build errors

- Run **cargo run --bin kassigner-setup** from the tools directory to check your environment.
- Ensure you're using the correct feature flags: default for Waveshare, **--no-default-features --features m5stack** for M5Stack.

LCD ghost image / burn-in

The ST7789T3 display retains charge from bright pixels (especially white QR code screens). This is not permanent burn-in — the firmware applies a gray wash sequence before camera init to discharge retained pixels. If ghosting persists, power cycle the device.

27. Command Reference

Firmware build commands

```
# Waveshare (default):
cd bootloader
ESP_HAL_CONFIG_PSRAM_MODE=octal cargo run --release

# M5Stack CoreS3:
cargo run --release --no-default-features --features m5stack

# Waveshare (dev, skip self-tests):
ESP_HAL_CONFIG_PSRAM_MODE=octal cargo run --release \
--features waveshare,skip-tests

# Flash only (no monitor):
espflash flash target/xtensa-esp32s3-none-elf/\
release/kassigner-bootloader
```

Clippy lint

```
ESP_HAL_CONFIG_PSRAM_MODE=octal cargo clippy \
--features waveshare,skip-tests
cargo clippy --no-default-features --features m5stack,skip-tests
```

Docker reproducible builds

```
docker build --platform linux/amd64 \
-f Dockerfile.base -t kassigner-toolchain:v1 .
docker build --platform linux/amd64 -t kassigner-build .
docker run --rm kassigner-build
```

Mirror viewer (separate terminal)

```
cd tools && cargo run --release --bin kassigner-mirror \
-- /dev/cu.usbmodem21201
```

Environment check

```
cd tools && cargo run --bin kassigner-setup && cd ..
```

KasSee companion

```
cd kassee && cargo build --release
```

Command	Description
`kassee import <kpub>`	Import watch-only public key
`kassee balance`	Show wallet balance
`kassee addresses [-n N]`	List receive addresses
`kassee addresses --change`	List change addresses
`kassee send <addr> <amt>`	Build unsigned KSPT, show as QR
`kassee send ... --fee N`	Set total fee in sompi (default 10000)
`kassee broadcast <hex>`	Broadcast signed transaction

What happens when you power off? Everything is gone. Every key, every seed, every passphrase — wiped from RAM. The device boots fresh every time. Your wallet lives in your backup. The device is just the tool that brings the wallet to life temporarily, signs what needs signing, and then forgets.

KasSigner — An InKasWeRust Project

kassigner@proton.me | github.com/InKasWeRust/KasSigner